Seminario de Arquitecturas de Software

Un Análisis Comparativo de Estilos Basados en Capas y Dominio

Maní Molido 2, ahora más molido

Integrantes

#	Nombre completo
1	Adrián Xavier Brioso Jurado
2	Alejandro García Ruiz
3	Ana Laura Hernández Cutiño
4	Juan Carlos Yern Espinosa
5	Laura Alonso Rivero

Contribución individual por pregunta

Integrante	Pregunta(s) en las que trabajó
Adrián Xavier Brioso Jurado	6
Alejandro García Ruiz	4
Ana Laura Hernández Cutiño	5
Juan Carlos Yern Espinosa	3
Laura Alonso Rivero	1, 2

Repositorio público del proyecto

`https://github.com/Jcyern/Management-for-the-automatic-generation-of-delivery-routes/tree/Entregas-IS`

23 de octubre de 2026

1. Análisis de Trade-offs (Arquitectura N-Capas vs. Clean / Hexagonal)

1.1. Diagnóstico del error fundamental de acoplamiento

Para resolver el problema del "monolito" y el acoplamiento, el equipo debe invertir su enfoque de diseño. En una arquitectura N-capas tradicional, las dependencias fluyen de la interfaz de usuario hacia la Base de Datos (UI → Lógica → Acceso a Datos → BD). En la práctica, la capa de negocio termina conociendo nombres, modelos, SQL/ORM de la base de datos, y la interfaz de usuario termina conteniendo o condicionando reglas de negocio. Si la BD cambia, la Lógica y la UI se ven afectadas.

Esto viola directamente la Regla de Dependencia de Martin la cual señala que los formatos de datos declarados en círculos externos no deben ser usados por círculos internos, y que nada externo debe impactar los círculos internos (Martin, p. 203). En la Arquitectura Hexagonal, Cockburn describe el mismo problema como el entrelazamiento entre la lógica de negocio y la interacción con entidades externas, y formula la regla: el código perteneciente a la parte interna no debe filtrarse hacia la parte externa (Cockburn, p. 3). La motivación original es justamente evitar que la lógica de negocio se infiltre en la interfaz y que la aplicación quede atada a la base de datos (Cockburn, p. 2).

En el sistema de gestión de inventario creado por este equipo, el error planteado "cambios en la base de datos afectan la lógica de negocio y la interfaz de usuario" indica que los detalles de persistencia están filtrándose hacia adentro, mientras q el síntoma "pruebas unitarias lentas y frágiles" sugiere que los casos de uso están acoplados a la UI/DB reales, por lo que no pueden probarse de forma aislada sin estos componentes. Eso es precisamente lo que las arquitecturas Clean/Hexagonal corrigen.

1.2. Cómo lo resuelve la Arquitectura Limpia (Clean Architecture)

La solución estructural es invertir las dependencias en las fronteras y dividir el software en capas concéntricas: Entidades, Casos de Uso, Adaptadores de Interfaz y Frameworks/Drivers (Martin, p. 204-205).

- Entidades: reglas de negocio críticas de la empresa, lo más general y estable, son las menos probables a cambiar (Martin, p. 204).
- Casos de Uso: reglas específicas de la aplicación; orquestan el flujo de los datos desde y hacia las entidades y no son afectados por cambios de DB, UI y frameworks comunes (Martin, p. 204).
- Adaptadores de Interfaz: convierten datos entre el formato más conveniente para casos de uso/entidades y el formato más conveniente para DB/web/UI; en esta capa se encuentra el MVC, presenters, views, controllers y todo el SQL (Martin, p. 205).
- Frameworks y Drivers: detalles externos como la base de datos y el framework web, esta compuesto por código que se conectara con el circulo interno (Martin, p. 205).

Aplicación: La lógica de negocio (Entidades y Casos de Uso) debe estar en el centro. La Base de Datos y la UI deben ir en los círculos más externos. La lógica de negocio nunca debe importar código de la base de datos ni de la interfaz de usuario.

1.3. Cómo lo resuelve la Arquitectura Hexagonal (Hexagonal Architecture)

Cockburn propone Puertos y Adaptadores: la aplicación está adentro y se comunica con el exterior mediante puertos; cada tecnología externa se conecta mediante un adaptador (Cockburn, p. 3-4). La aplicación no sabe si la maneja un usuario, un test automatizado, un script batch u otro programa (Cockburn, p. 2).

Se distinguen puertos primarios (actores que manejan la aplicación) y secundarios (actores que la aplicación maneja) (Cockburn, p. 8-9). También indica que los casos de uso deben escribirse en la frontera de la aplicación, sin conocimiento íntimo de la tecnología externa (Cockburn, p. 9).

- Puerto primario/driving: interfaz para los casos de uso. La UI web, un test harness o un proceso batch son adaptadores que lo invocan, se encontraran en la parte izquierda o la cima del hexágono
- Puerto secundario/driven: interfaz de repositorio. SQL Server y MongoDB son adaptadores que lo implementan, se encontraran en la parte derecha o el fondo
- La lógica de negocio queda en el hexágono interno; los adaptadores convierten llamadas y datos.

1.4. Impacto en la Testabilidad

Las pruebas son lentas y frágiles porque dependen de la base de datos real y de detalles de la UI. Con Arquitectura Clean/Hexagonal, la aplicación puede ejecutarse sin UI y sin base de datos real por lo que el impacto es altamente positivo. Al aplicar la Regla de Dependencia y los Puertos, la lógica de negocio queda aislada. Como menciona Cockburn, se pueden usar "test harnesses" y "mock databases" (Cockburn, p. 5). Martin también señala que separar el software en capas permite crear "un sistema que es intrínsecamente testeable" (Martin, p. 209). Las pruebas unitarias se ejecutarán contra los Casos de Uso utilizando adaptadores falsos, sin necesidad de levantar la base de datos real, haciéndolas rápidas y estables.

1.5. Impacto de cambiar la Base de Datos de SQL Server a MongoDB

Con la Arquitectura N-capas cambiar la DB rompe la lógica de negocio mientras que con Arquitectura Clean/Hexagonal el impacto es mínimo y localizado.

Según Martin, en la capa de Adaptadores de Interfaz, si la base de datos es una base de datos SQL, entonces todo el SQL debe restringirse a esta capa (Martin, p. 205). Si el equipo decide migrar a MongoDB, solo necesita crear un nuevo Adaptador (en la capa

externa de frameworks o en el lado derecho del hexágono) que implemente el mismo puerto (interfaz) que la aplicación ya espera, no va a cambiar aunque cambie la tecnología (Cockburn, p. 3-4).

La lógica de negocio (Casos de Uso y Entidades) no se toca. No hay necesidad de reescribir reglas de negocio, ya que la aplicación simplemente envía y recibe estructuras de datos simples a través del puerto, ignorando por completo si detrás hay SQL Server o MongoDB.

2. Comparación de Estilos: La Simetría del Hexágono vs. La Jerarquía de las Capas

2.1. ¿Qué problema conceptual resuelve la simetría del hexágono que una representación apilada no transmite?

La representación en capas apiladas —tanto la N-Capas tradicional como la Onion— transmite intuitivamente una idea de jerarquía vertical: arriba la UI, en medio la lógica de negocio, abajo el acceso a datos. Lo que sugiere que cada capa sólo habla con la inmediatamente inferior, y que la base de datos es el "fundamento" del sistema. Palermo critica exactamente eso: en la arquitectura en capas tradicional la lógica del negocio termina acoplada con las infraestructuras como Data Access, I/O y Web Services, las cuales cambian con frecuencia por lo que sería absurdo hacer una acoplación tan estrecha a WCF o al acceso a datos (Palermo, 2008c, p. 1). La conclusión de Palermo es que la infraestructura debe empujarse a los bordes y el núcleo debe depender de abstracciones, no del código concreto de acceso a datos (Palermo, 2008c, p. 1-2).

Estas arquitectura no representa bien más de dos puertos. Muchas aplicaciones tienen sólo dos puertos diálogo del lado usuario y diálogo del lado base de datos, lo que les da una apariencia asimétrica y hace parecer natural construir el sistema como una arquitectura apilada de tres, cuatro o cinco capas. Pero hay dos problemas: primero, la gente no se toma en serio las "líneas" del dibujo en capas y deja que la lógica de la aplicación se filtre a través de las fronteras; segundo, puede haber más de dos puertos, y entonces la arquitectura no encaja en el dibujo unidimensional (Cockburn, p. 4).

La simetría del hexágono resuelve ese problema conceptual porque obliga a pensar el sistema como: un núcleo de aplicación adentro que se comunica a través de un número determinado de puertos con cosas afuera; los elementos externos pueden tratarse simétricamente (Cockburn, p. 4).

Es decir, la UI no es "la capa superior" ni la base de datos "la capa inferior": ambas son agencias externas conectadas por adaptadores al mismo tipo de frontera. El hexágono destaca:

- la asimetría dentro-fuera (el núcleo no debe conocer los detalles externos)
- la naturaleza similar de los puertos (todos son conversaciones con un propósito, no jerarquías tecnológicas)
- la posibilidad de múltiples adaptadores por puerto: GUI, test harness, batch, HTTP, mock DB, base real, etc. (Cockburn, p. 4).

En cambio, la representación de capas apiladas tiende a naturalizar una dependencia transitiva UI → negocio → datos, y a esconder que la base de datos es un detalle reemplazable. Palermo intenta corregir esto en Onion: la regla fundamental es que todo código puede depender de las capas centrales pero no de las capas más alejadas del núcleo; el modelo de dominio está al centro y la infraestructura queda en los bordes (Palermo, 2008a, p. 2). Pero incluso así, el dibujo concéntrico sigue transmitiendo una idea de jerarquía y de "capas hacia el centro"; el hexágono comunica mejor la equivalencia arquitectónica de los puertos y la sustituibilidad de los adaptadores.

Palermo mismo reconoce que Onion y Hexagonal comparten la premisa de externalizar la infraestructura y escribir código adaptador para que la infraestructura no quede fuertemente acoplada (Palermo, 2008a, p. 2). La diferencia es de énfasis visual y conceptual: Onion enfatiza la dirección del acoplamiento hacia el centro mientras que Hexagonal enfatiza la frontera dentro-fuera y la simetría de los puertos.

2.2. Cómo se traduce "Puerto y Adaptador" en la práctica de definir interfaces en las capas internas de Onion

En Hexagonal, un puerto es una conversación con un propósito definido, expresada como una API o interfaz; un adaptador es el código específico de tecnología que convierte las señales de un dispositivo externo a esa API y viceversa. La aplicación no sabe qué tecnología hay al otro lado: solo conoce el puerto (Cockburn, 2005, p. 2-4). En Onion no se usa siempre el término "puerto", pero la práctica es equivalente:

- Puerto = interfaz definida en una capa interna, que expresa una conversación con un propósito desde el punto de vista del núcleo. En Onion, la primera capa alrededor del dominio suele contener las interfaces de repositorio; sólo la interfaz está en el núcleo, no la implementación, porque guardar/recuperar objetos implica una base de datos (Palermo, 2008a, p. 2).
- Adaptador = clase en una capa externa que implementa esa interfaz con una tecnología concreta: SQL Server, MongoDB, WCF, archivo, UI, test harness, mock, etc. Esa clase está acoplada a un método concreto de acceso a datos, y por eso reside en la periferia (Palermo, 2008a, p. 2).

El ejemplo de Palermo es exacto: SpeakerController pertenece a la UI y depende de IConferenceRepository e IUserSession, interfaces definidas en el núcleo de la aplicación; ConferenceRepository y UserSession están en capas externas e implementan esas interfaces. En tiempo de ejecución, el contenedor IoC resuelve las implementaciones y las inyecta en el constructor del controlador (Palermo, 2008b, p. 1-2). Eso es Puertos y Adaptadores con otro dibujo: la interfaz interna es el puerto; la clase externa es el adaptador; la inversión de dependencias/IoC es el mecanismo de conexión.

Palermo lo formula como principios de Onion:

- el modelo de objetos es independiente
- las capas internas definen interfaces mientras que las capas externas las implementan
- la dirección del acoplamiento es hacia el centro

- todo el código del núcleo puede compilarse y ejecutarse separado de la infraestructura (Palermo, 2008c, p. 3).

Por tanto, en Onion, definir IConferenceRepository en el núcleo es definir un puerto; ConferenceRepository es un adaptador; un MockConferenceRepository para tests es otro adaptador.

2.3. Por qué Cockburn dice que el hexágono no tiene seis lados por una razón numérica

Cockburn lo dice literalmente: El seis es arbitrario; la forma poligonal solo da espacio visual para dibujar varios puertos y varios adaptadores por puerto, sin estar limitados por capas unidimensionales (Cockburn, 2005, p. 4). Cockburn ha visto aplicaciones con dos, tres o cuatro puertos; el hexágono permite representar esa multiplicidad sin volver a una pila unidimensional (Cockburn, 2005, p. 9-10). La forma sirve para eliminar la asimetría izquierda-derecha/arriba-abajo: en el nivel arquitectónico, todos los puertos son fundamentalmente similares; la aplicación está "dentro" y las cosas externas están "fuera", sin que la UI sea superior ni la base de datos inferior. Cockburn reconoce que en la implementación aparecen puertos primarios —actores que dirigen la aplicación— y secundarios —actores a los que la aplicación dirige—, y que por convención se dibujan a izquierda/cima y derecha/fondo; pero aclara que el patrón está escrito deliberadamente pretendiendo que todos los puertos son fundamentalmente similares, y esa pretensión es útil en el nivel arquitectónico (Cockburn, 2005, p. 8-9). La distinción primario/secundario es una consecuencia de implementación, no una jerarquía que deba deformar la arquitectura

3. Aplicación de DDD y Bounded Context en un Dominio Complejo

3.1. ¿Cómo dividiría el sistema en contextos delimitados?

La división del sistema se plantea a partir de las capacidades del negocio y del lenguaje que utiliza cada grupo de expertos, no a partir de los departamentos de la empresa ni del modelo de datos. Siguiendo el enfoque estratégico de DDD, se identifican primero los subdominios y, a partir de ellos, se delimitan los contextos en los que cada modelo tiene un significado propio.

Un *Bounded Context* es una frontera explícita dentro de la cual un modelo y su lenguaje ubicuo son válidos; fuera de ella, el mismo término puede tener otro significado (Evans, 2015, p. 1-2; Richards & Ford, 2020, cap. 7). El lenguaje ubicuo es el vocabulario compartido por expertos y desarrolladores dentro de esa frontera, y deja de ser válido fuera de ella (Evans, 2015, p. 2).

El enunciado menciona tres subsistemas (siniestros, facturación y ajustes), pero la gestión de la póliza y la identidad de los participantes son capacidades necesarias para que esos subsistemas funcionen; por eso se incorporan como contextos propios. Los subdominios identificados son los siguientes:

- **Suscripción y emisión de pólizas**: subdominio núcleo. Define el contrato, las cláusulas, las coberturas, la vigencia y las reglas de emisión y endosos.
- **Gestión de siniestros**: subdominio núcleo. Gestiona el reporte, el expediente, el estado, la asignación y la resolución del siniestro.
- **Evaluación de daños en campo**: subdominio de apoyo. Corresponde a la aplicación que utilizan los ajustadores en el lugar del accidente: evidencias, fotografías, ubicación y estimación del daño.
- **Facturación de primas**: subdominio de apoyo. Gestiona primas, facturas, pagos, cobranza y mora.
- **Clientes**: subdominio genérico. Identidad de tomadores, asegurados y beneficiarios.
- **Catálogo de coberturas y tarifas**: subdominio de apoyo. Productos, coberturas, tarifas y plantillas de cláusulas; puede formar parte de Pólizas o separarse si existen varios productos y canales.

La distinción entre subdominio núcleo, de apoyo y genérico sigue la clasificación estratégica de DDD: el núcleo concentra la ventaja competitiva, los de apoyo sostienen al núcleo sin ser diferenciadores y los genéricos pueden resolverse con soluciones comunes (Evans, 2015, p. 40-41). A partir de estos subdominios se definen los siguientes contextos delimitados:

Contexto	**Tipo**	**Significado de "póliza"**	**Modelo y agregados**
Pólizas / Suscripción	Núcleo	El contrato con cláusulas, coberturas y vigencia	`Policy` (raíz), `Coverage`, `Clause`, `ValidityPeriod`, `Premium`
Siniestros	Núcleo	Una referencia y la cobertura vigente al momento del siniestro	`Claim` (raíz), `ClaimNumber`, `PolicyReference`, `CoverageSnapshot`, `DamageItem`
Ajustes de campo	Apoyo	Un número de referencia y los datos necesarios para evaluar el daño	`Appraisal` (raíz), `DamageAssessment`, `Evidence`, `GeoLocation`, `AdjusterId`
Facturación de primas	Apoyo	Una fuente de ingresos y un calendario de pagos	`Invoice` (raíz), `BillingAccount`, `Premium`, `Payment`, `PaymentSchedule`
Clientes	Genérico	El asegurado, tomador o beneficiario	`Customer` (raíz), `Document`, `Contact`
Catálogo de coberturas	Apoyo	El producto y la cobertura contratable	`CoverageProduct` (raíz), `RateTable`, `ClauseTemplate`

Cuadro 1: Contextos delimitados propuestos para el sistema de seguros.

La diferencia de significado es el argumento central de la división: "póliza" no es una sola cosa. Para el actuario es un contrato con cláusulas; para siniestros es una referencia con una cobertura vigente; para facturación es una fuente de ingresos; para el ajustador es un número de referencia. Cada contexto define su propio modelo y reconcilia las diferencias en los puntos de integración, en lugar de imponer una entidad única para toda la organización (Richards & Ford, 2020, cap. 7).

La delimitación se justifica con criterios de granularidad de servicios y de lenguaje ubicuo:

- **Propósito**: cada contexto tiene una responsabilidad cohesionada. Pólizas no calcula indemnizaciones; Facturación no interpreta cláusulas; Ajustes no emite contratos.

- **Transacciones**: cada agregado es una frontera de consistencia. La emisión de una póliza, el reporte de un siniestro y la generación de una factura son transacciones distintas; no conviene una transacción distribuida entre las tres (Evans, 2015, p. 15).
- **Coreografía**: Ajustes se separa porque tiene un actor, un canal y un ciclo de vida propios. Si la comunicación entre Ajustes y Siniestros fuera excesiva, podría fusionarse con Siniestros; la división es una primera iteración y debe validarse con los expertos.
- **Lenguaje y equipos**: cada contexto puede tener su propio lenguaje ubicuo y, potencialmente, su propio equipo, lo que reduce la coordinación y permite evolucionar cada modelo a distinto ritmo (Evans, 2015, p. 2; Richards & Ford, 2020, cap. 7).

Es importante aclarar que la aplicación del ajustador no es un contexto por ser una aplicación. Sería solo un adaptador de interfaz si no tuviera modelo propio. En este caso sí lo tiene, porque la evaluación de daños posee identidad, evidencia, reglas y estados propios; por eso se justifica como contexto delimitado.

Finalmente, la división no se plantea por capas técnicas (controladores, servicios, repositorios), porque esas son capas dentro de un contexto, no contextos delimitados. Tampoco se asume que cada departamento sea automáticamente un contexto: la frontera se define por el modelo y el lenguaje, no por el organigrama. Los contextos propuestos son lógicos y pueden implementarse como módulos de un mismo monolito o como servicios separados. El *Bounded Context* se relaciona habitualmente con los microservicios, pero el concepto no obliga a distribuir: los contextos pueden convivir en un mismo despliegue siempre que cada uno conserve su modelo y su frontera (Richards & Ford, 2020, cap. 17).

3.2. ¿Cómo evitaría que el modelo de "Póliza" se contamine con detalles de siniestros o facturación?

Para evitar la contaminación, la regla es no compartir la entidad `Policy` entre contextos. Cada contexto define su propia representación de la póliza de acuerdo con el lenguaje y las decisiones que le competen:

- En Pólizas, `Policy` es el agregado completo: cláusulas, coberturas, vigencia y reglas de emisión y endosos.
- En Siniestros, no se importa `Policy`; se guarda un `PolicyReference` y, si se necesita, un `CoverageSnapshot` con la cobertura vigente al momento del siniestro.
- En Facturación, la póliza se representa como una referencia financiera y un calendario de pagos; no interesan las cláusulas.
- En Ajustes, solo se conserva el número de referencia y los datos del vehículo y la cobertura necesarios para evaluar el daño.

Se prefiere duplicar la referencia mínima antes que compartir la entidad completa: cuando se busca un alto desacoplamiento, la duplicación suele ser preferible a la reutilización, porque la reutilización entre contextos introduce acoplamiento (Richards & Ford, 2020, cap. 17).

La separación se protege en ambos sentidos: Pólizas no conoce `Claim` ni `Invoice`, y Siniestros, Ajustes y Facturación no conocen las cláusulas del contrato.

La separación se refuerza con las siguientes decisiones de diseño:

- **Referencias por identidad, no por objeto**: los contextos se comunican con identificadores como `PolicyId`, `PolicyNumber` o `ClaimId`; nunca con una referencia directa a la clase `Policy` de otro contexto.
- **Anticorruption Layer**: cada contexto que consume información de Pólizas traduce el DTO (objeto de transferencia de datos) o el evento recibido a su propio modelo. Así, el modelo externo no entra en el dominio local (Evans, 2015, p. 34).
- **Published Language**: los contratos de integración se publican y versionan; los eventos y DTOs llevan solo los datos necesarios, no el agregado completo (Evans, 2015, p. 36).
- **Datos propios**: cada contexto es dueño de su esquema o base de datos; no hay tablas compartidas ni un `Policy` central al que todos accedan.
- **Invariantes dentro del agregado**: las reglas de la póliza se validan en `Policy`; las del siniestro, en `Claim`; las de la factura, en `Invoice`. Solo la raíz del agregado es accesible desde fuera y solo ella tiene repositorio (Evans, 2015, p. 15).
- **Sin Shared Kernel de `Policy`**: a lo sumo se comparte el contrato publicado de `PolicyNumber` y `Money`, y siempre como lenguaje de integración, no como clase de dominio compartida.

Con esto, un cambio en las cláusulas o en las reglas de emisión no obliga a modificar Siniestros ni Facturación, y un cambio en la facturación no obliga a modificar el contrato. Cada contexto conserva solo la parte de la póliza que necesita, lo que es precisamente la opacidad que define un *Bounded Context* (Evans, 2015, p. 1; Richards & Ford, 2020, cap. 7).

3.3. ¿Cómo se comunicarían estos contextos de manera desacoplada?

La comunicación se resuelve con dos mecanismos complementarios: eventos de integración asíncronos y consultas síncronas puntuales a través de un Open Host Service.

- **Eventos de integración**: los contextos publican hechos por un bus de mensajes, con solo los identificadores y datos relevantes; esto introduce consistencia eventual, pero evita el acoplamiento temporal. Los principales eventos son, por ejemplo:
  - Pólizas: `PolicyIssued`, `PolicyEndorsed`, `PolicyCancelled`.
  - Siniestros: `ClaimReported`, `ClaimAssigned`, `ClaimResolved`.
  - Ajustes: `AppraisalCompleted`.
  - Facturación: `PremiumInvoiced`, `PaymentReceived`.

- **Consultas síncronas**: cuando Siniestros necesita verificar la vigencia y las coberturas en el momento del siniestro, Pólizas expone un Open Host Service para consultar la cobertura vigente de una póliza en una fecha determinada, con un Published Language estable (Evans, 2015, p. 35-36). Siniestros consume esa respuesta a través de un adaptador con Anticorruption Layer.

- **Sin base de datos compartida**: los contextos no se integran por tablas comunes ni por acceso directo a los modelos; cada uno mantiene su propio almacenamiento.

- **Sin transacciones distribuidas**: cada contexto confirma su propia transacción y, cuando un proceso cruza varios contextos, la capa de aplicación coordina los pasos mediante eventos y compensaciones, en lugar de una transacción única entre Pólizas, Siniestros y Facturación.

El mapa de contextos queda así: Pólizas es *upstream* de Siniestros y Facturación y les publica eventos; Siniestros y Ajustes cooperan mediante eventos; Facturación consume eventos de Pólizas, pero no accede a su modelo; Clientes se expone como Open Host Service genérico para los demás contextos (Evans, 2015, p. 27).

Esta forma de comunicación es desacoplada porque los contextos solo dependen de contratos publicados, no de clases, tablas ni tecnologías de los demás. Un cambio interno en Siniestros no obliga a cambiar Pólizas, y cambiar el bus de mensajes o la base de datos de un contexto solo afecta a sus adaptadores.

3.4. ¿Cómo se relaciona esta solución con Onion/Clean y con el Adaptador de Servicio?

Cada contexto delimitado puede implementarse como su propio hexágono u onion. Dentro de cada uno, la Regla de Dependencia se mantiene: el dominio no conoce la base de datos, ni HTTP, ni el bus de mensajes; las dependencias apuntan hacia adentro (Martin, p. 203; Palermo, 2008c, p. 3).

- **Dominio**: agregados, objetos de valor, servicios de dominio, interfaces de repositorio y eventos de dominio. Es el corazón del contexto y no depende de infraestructura.

- **Aplicación**: servicios de aplicación y casos de uso como emitir una póliza, reportar un siniestro, evaluar un daño o facturar una prima; coordina transacciones y procesos, pero no contiene reglas de negocio.

- **Infraestructura/adaptadores**: repositorios concretos, publicadores de eventos, clientes HTTP y adaptadores de servicio. Implementan los puertos definidos hacia adentro.

El **Adaptador de Servicio** es la pieza que conecta un contexto con otro o con un servicio externo. En la Arquitectura Hexagonal es un adaptador de un puerto de salida (Cockburn, p. 3-4); en la arquitectura N-Capas su equivalente es el **Agente de Servicios externos**, que aísla la semántica específica del servicio externo, mapea los formatos de datos y permite sustituir el servicio sin que el sistema se vea afectado (De la Torre et al., 2010, p. 155-156). Por ejemplo:

- En Siniestros, el puerto `IPolicyQueryPort` vive en la capa de aplicación; el adaptador `PolicyServiceAdapter` vive en infraestructura, llama al Open Host Service de Pólizas y traduce el DTO externo a `CoverageSnapshot`. El dominio de Siniestros nunca ve el modelo de Pólizas.
- En Facturación, un adaptador traduce el evento de emisión de póliza y actualiza su propio `BillingAccount`. El dominio de Facturación no conoce la clase `Policy`.
- En Pólizas, el puerto de publicación de eventos es `IEventPublisher`. Un adaptador concreto para Kafka o RabbitMQ lo implementa; cambiar el bus no afecta al dominio.

Esta separación produce tres beneficios: el dominio se puede probar con adaptadores falsos, sin HTTP ni base de datos; cambiar un proveedor o una tecnología solo afecta al adaptador; y cada contexto puede evolucionar a su propio ritmo. Es la aplicación conjunta de la Regla de Dependencia, los puertos y adaptadores y el Agente de Servicios externos (Martin, p. 203; Cockburn, p. 3-4; De la Torre et al., 2010, p. 155-156).

4. Evaluación de la Inversión de dependencias en la práctica

4.1. a) ¿Por qué esta dependencia directa viola los principios fundamentales de las arquitecturas Hexagonal, Limpia y Onion? ¿Qué dirección deberían tener las dependencias y por qué no se cumple?

Esta dependencia directa viola los principios fundamentales porque el núcleo de la aplicación (lógica de negocio, OrderProcessor) está acoplado a un detalle de infraestructura volátil (el servicio de validación y la librería HTTP). Al instanciar directamente AddressValidatorService y depender de su librería HTTP, el código fuente de alto nivel depende de código de bajo nivel.

Dirección que deberían tener las dependencias:

Según Martin, la Regla de Dependencia exige que las dependencias del código fuente apunten solo hacia adentro, hacia las políticas de mayor nivel. Nada en un círculo interno puede saber absolutamente nada sobre algo en un círculo externo (Martin, p. 203). El Principio de Inversión de Dependencias (DIP) refuerza esto indicando que las dependencias del código fuente deben referirse solo a abstracciones, no a concreciones volátiles (Martin, p. 87-89).

Por qué la implementación actual no cumple:

- Arquitectura Hexagonal: Cockburn establece que “el código perteneciente a la parte interna no debe filtrarse hacia la parte externa” (Cockburn, p. 3). Aquí ocurre lo contrario: el exterior contamina al interior. La aplicación debe ser “felizmente ignorante” de la naturaleza de los dispositivos de entrada/salida (Cockburn, p. 2).
- Clean Architecture: OrderProcessor (núcleo) menciona el nombre de algo declarado en el círculo externo (AddressValidatorService), violando directamente la regla (Martin,

p. 203). Además, los detalles externos (la web, los servicios, la base de datos) deben mantenerse afuera donde pueden hacer poco daño (Martin, p. 205).

- Arquitectura Onion: Palermo indica que la dirección del acoplamiento debe ser hacia el centro y el modelo de objetos debe ser independiente (Palermo, 2008c, p. 3). Además, advierte que acoplar la lógica de negocio a la infraestructura (como servicios web o acceso a datos) es un error, ya que la infraestructura cambia frecuentemente y el núcleo debe sobrevivir a esos cambios (Palermo, 2008c, p. 1).

4.2. b) Rediseño de la interacción entre OrderProcessor y AddressValidatorService

Para alinear el diseño con las arquitecturas, debemos aplicar el Principio de Inversión de Dependencias (DIP) y el patrón de Puertos y Adaptadores.

El nuevo diseño sería el siguiente:

1. En el núcleo de la aplicación/ interior del hexágono):
   - Se define una interfaz (o puerto), por ejemplo: IAddressValidator (o AddressValidatorPort).
   - Esta interfaz debe contener el método que el negocio necesita
   - OrderProcessor ahora depende únicamente de IAddressValidator. Ya no conoce ni instancia AddressValidatorService ni la librería HTTP.
2. En la capa de infraestructura/ exterior del hexágono (Adaptador):
   - Se crea una clase, por ejemplo: HttpAddressValidatorAdapter (que sería la evolución de AddressValidatorService).
   - Esta clase implementará la interfaz IAddressValidator y contendrá toda la lógica de comunicación HTTP y la dependencia de la librería externa.

Martin explica en su Capitulo 11 de DIP que las interfaces (abstracciones) son menos volátiles que las implementaciones (detalles). Un buen diseño evita depender de concreciones volátiles y favorece el uso de interfaces estables (Martin, p. 89).

Basándonos en el principio de que "los detalles deben depender de las abstracciones" podemos decir que en el rediseño:

- La Abstracción (Interfaz IAddressValidator): Representa la necesidad del negocio. El negocio necesita saber si una dirección es válida. Esta interfaz es sumamente estable, porque la regla de negocio ("validar dirección") no cambiará aunque la tecnología subyacente cambie. Por lo tanto, la abstracción vive en el núcleo.
- El Detalle (HttpAddressValidatorAdapter): Representa la implementación técnica (la librería HTTP). Es altamente volátil. Este detalle vive en la infraestructura (afuera) y su código debe "apuntar" hacia adentro, porque debe implementar la interfaz IAddressValidator.

OrderProcessor ya no depende de AddressValidatorService. Ahora, depende de IAddressValidator. Y HttpAddressValidatorAdapter también depende de IAddressValidator (porque lo implementa). Como resultado, la flecha de dependencia del código fuente se ha invertido: tanto el núcleo como la infraestructura apuntan hacia la abstracción, pero el núcleo ya no sabe nada del exterior.

Desde Clean Architecture podemos justificar el rediseño porque cuando el flujo de control va hacia afuera (el negocio necesita validar una dirección en internet), usamos el Principio de Inversión de Dependencias (DIP). El caso de uso (o el procesador) llama a una interfaz interna, y la implementación externa (el adaptador HTTP) la implementa. Así, las dependencias del código fuente se oponen al flujo de control (Martin, p. 206-207).

Desde la Arquitectura Hexagonal podemos justificar el rediseño porque IAddressValidator actúa como un Puerto. Es una API que define el propósito de la conversación. HttpAddressValidatorAdapter es el Adaptador, que convierte las llamadas del puerto en señales HTTP entendibles por el servicio externo. El núcleo está "felizmente ignorante" de si el adaptador usa HTTP, un archivo de texto o cualquier otro método (Cockburn, p. 2-3).

Desde la Arquitectura Onion podemos justificar el rediseño porque Palermo dicta como principio fundamental que "las capas internas definen interfaces, las capas externas las implementan" y que "el acoplamiento se dirige hacia el centro" (Palermo, 2008c, p. 3). Al poner IAddressValidator en el núcleo, garantizas que todo el código principal pueda compilarse y probarse sin necesidad de tener la librería HTTP instalada o conectividad a internet (Palermo, 2008c, p. 3).

4.3. c) Implicaciones del rediseño para la mantenibilidad y la capacidad de cambiar el proveedor de validación de direcciones en el futuro

Gracias al rediseño, OrderProcessor recibe a través de su constructor una instancia que cumpla con el contrato de IAddressValidator. Durante las pruebas unitarias, se puede inyectar un mock de la interfaz, en lugar de la implementación real. Esto evita hacer llamadas HTTP reales (que son lentas, costosas y dependen de la red), permitiendo que las pruebas se ejecuten en milisegundos y de forma determinista.

- Cockburn (Hexagonal) enfatiza que el patrón permite que la aplicación sea "desarrollada y probada de forma aislada de sus eventuales dispositivos de tiempo de ejecución y bases de datos", sustituyendo actores secundarios reales por mocks para ejecutar pruebas de regresión automatizadas sin intervención humana (Cockburn, p. 2, 3 y 8).
- Martin (Clean) concluye que separar el software en capas y respetar la Regla de Dependencia crea un sistema que es "intrínsecamente testeable, con todos los beneficios que eso implica" (Martin, p. 209).

El rediseño desacopla la lógica de negocio (el núcleo) de los detalles de infraestructura (el exterior), cumpliendo con el Principio de Inversión de Dependencias (DIP), el cual dicta que las dependencias del código fuente deben apuntar a abstracciones y no a concreciones volátiles (Martin, p. 88-89).

- Si la librería HTTP subyacente cambia (por ejemplo, de una librería obsoleta a una moderna), solo se modifica el adaptador (HttpAddressValidatorAdapter) en la capa externa. OrderProcessor (el núcleo) no se toca.
- Si se decide cambiar de proveedor de validación, simplemente se crea un nuevo adaptador que implemente IAddressValidator y se configura su inyección. El código del núcleo permanece inmutable.
- Con Clean Architecture se afirma que, al cumplir la Regla de Dependencia, "cuando cualquiera de las partes externas del sistema quede obsoleta, como la base de datos o el framework web, puede reemplazar esos elementos obsoletos con un mínimo de complicaciones" (Martin, p. 209).
- Con Onion Architecture se agrega que este enfoque asegura que el núcleo de la aplicación no tenga que cambiar a medida que [...] cambia el acceso a datos, los servicios web y la infraestructura de mensajería (Palermo, 2008c, p. 2), resultando en sistemas de larga vida y fáciles de mantener, donde el código es muy fácil de modificar (Palermo, 2008a, p. 1).

El diseño transforma un sistema rígido en uno flexible. El OrderProcessor queda protegido como un núcleo blindado: altamente testeable sin internet, y totalmente resiliente a los cambios tecnológicos del exterior.

5. Análisis de un Problema Real: El "Kitty Problem" y el Principio de Abierto/Cerrado

5.1. Aclaración sobre la fuente

La orientación del seminario atribuye el *Kitty Problem* a Richards y Ford (2020). Al contrastarlo con la bibliografía, el ejemplo aparece en Martin (2017, cap. 27, pp. 242-246), dentro de *Clean Architecture*, no en *Fundamentals of Software Architecture*. Richards y Ford aportan el marco de *bounded context* y de partición por dominio (Richards & Ford, 2020, caps. 7 y 8), que aquí se usa como apoyo conceptual, pero el caso concreto y su solución son de Martin. Se cita a Martin como fuente primaria del ejemplo y a Richards y Ford como apoyo para los conceptos de contexto delimitado y de fronteras arquitectónicas.

5.2. El problema en concreto

Martin (2017, pp. 242-243) describe un agregador de taxis compuesto por cuatro microservicios, cada uno asignado a un equipo: `TaxiUI` (atiende a los clientes), `TaxiFinder` (examina los inventarios de los proveedores y propone candidatos), `TaxiSelector` (aplica los criterios del usuario y elige un taxi) y `TaxiDispatcher` (ordena el taxi seleccionado). El sistema funciona durante un año con normalidad.

Luego el departamento de *marketing* anuncia un servicio de entrega de gatitos a domicilio. El nuevo requisito introduce reglas que no existían: solo algunos proveedores participan; los conductores alérgicos no deben ser seleccionados; un vehículo que ha transportado gatitos en los últimos tres días no debe asignarse a clientes con alergia declarada; y la

recogida debe pasar por un punto de recogida de gatitos antes de la entrega. Martin plantea entonces una pregunta importante: ¿cuántos servicios deben cambiar? Su respuesta es todos (Martin, p. 243). El desarrollo y el despliegue de la nueva funcionalidad tendrán que coordinarse entre todos los equipos, lo que demuestra que los servicios están acoplados y no pueden desarrollarse, desplegarse ni mantenerse de forma independiente.

5.3. Por qué falló la descomposición por verbos

La causa es cómo se trazaron las fronteras, no que los servicios fueran "demasiado pequeños". Los servicios se dividieron según pasos del proceso, es decir, según verbos. Cada paso contiene implícitamente el conocimiento de que lo que se transporta son personas. Cuando aparece un nuevo tipo de carga (gatitos), cambia la semántica de todos los pasos a la vez: encontrar ahora implica filtrar proveedores participantes y conductores alérgicos; seleccionar implica considerar el historial reciente del vehículo; despachar implica pasar por un punto de recogida. La variación del negocio (qué se transporta) atraviesa perpendicularmente la descomposición elegida (qué paso del proceso es).

Martin (2017, p. 243) lo formula como el problema de las preocupaciones transversales (*cross-cutting concerns*): las descomposiciones funcionales son muy vulnerables a nuevas funcionalidades que cortan transversalmente todos esos comportamientos. A ello se suman dos ideas:

- La falacia del desacoplamiento (Martin, p. 241): creer que dos componentes están desacoplados porque se ejecutan en procesos distintos. Si comparten datos o comportamiento, su desarrollo, despliegue y operación deben coordinarse igualmente.
- La trampa de la entidad (Richards & Ford, 2020, cap. 8): diseñar componentes como correspondencia con funciones técnicas o entidades en lugar de con flujos de negocio reales. El resultado son componentes que parecen especializados pero que no encapsulan la variación del dominio.

En el caso de los taxis, el concepto de negocio que debía haberse modelado no era el viaje de personas, sino uno más general. Por ejemplo, un servicio de transporte del que el viaje de pasajeros y la entrega de gatitos fuesen variantes. Al no modelarlo, la variación quedó repartida entre los cuatro servicios.

5.4. La solución: núcleo estable y componentes enchufables

Martin (2017, p. 244) propone resolver el problema con una arquitectura basada en componentes guiada por los principios SOLID, de modo que las nuevas funcionalidades se añadan sin modificar los componentes existentes. La estrategia es:

1. Conservar en clases base abstractas la lógica genérica de los servicios originales: filtrar candidatos, seleccionar y despachar un vehículo.
2. Extraer la lógica específica de los viajes de personas a un componente `Rides`.
3. Colocar la nueva funcionalidad en un componente `Kittens`.

4. Permitir que ambos componentes extiendan las abstracciones del núcleo mediante patrones como *Template Method* o *Strategy*, respetando la Regla de Dependencia: los componentes externos dependen de las abstracciones del núcleo, nunca al revés.

5. Crear las clases concretas mediante fábricas controladas por la interfaz de usuario.

Un esbozo mínimo del núcleo y de los dos componentes sería así:

```
// Núcleo estable: abierto a extensión, cerrado a modificación
public interface ICandidateFilter {
    bool Accepts(Vehicle v, TransportRequest r);
}

public interface IDispatchPlan {
    IEnumerable<Stop> BuildRoute(TransportRequest r);
}

public abstract class TransportService {
    protected abstract IEnumerable<ICandidateFilter> Filters { get; }
    protected abstract IDispatchPlan Plan { get; }

    public Vehicle Handle(TransportRequest request,
                          IEnumerable<Vehicle> fleet) {
        var candidate = fleet.First(
            v => Filters.All(f => f.Accepts(v, request)));
        candidate.AssignRoute(Plan.BuildRoute(request));
        return candidate;
    }
}

// Componente Rides: comportamiento original extraído
public class RideService : TransportService {
    /* filtros de lujo, coste, etc. */
}

// Componente Kittens: se despliega como una DLL adicional
public class KittenDeliveryService : TransportService {
    protected override IEnumerable<ICandidateFilter> Filters =>
        new ICandidateFilter[] {
            new ParticipatingSupplierFilter(),
            new DriverNotAllergicFilter()
        };

    protected override IDispatchPlan Plan =>
        new ViaCollectionPointPlan();
}
```

El resultado, según Martin (2017, p. 244), es que al implementar la funcionalidad de gatitos solo cambia `TaxiUI`; nada más necesita modificarse. Se añade un nuevo `.jar`, `.dll` o `Gem` al sistema y se carga en tiempo de ejecución. En la práctica, esto convierte lo que antes era un cambio coordinado entre cuatro equipos en un cambio localizado en la interfaz de usuario y un nuevo artefacto desplegable.

5.5. Composición de filtros, no solo *plugins*

La solución no se reduce a añadir `Kittens`. Algunas de las reglas transversales que introduce el nuevo requisito no forman parte intrínseca del núcleo, sino que son políticas que pueden componerse sobre él. Por ejemplo, que un vehículo que ha transportado gatitos en los últimos tres días no se asigne a clientes alérgicos. En el esbozo anterior, esa regla se implementa como un `ICandidateFilter` adicional que se aplica a los clientes con alergia declarada sin modificar `Rides` ni `Kittens`. El núcleo abstracto no solo permite añadir componentes completos, sino también componer reglas dentro de un mismo componente. Ese diseño es el que hace que el sistema sea verdaderamente extensible y no solo ampliable con otro servicio.

5.6. Relación con OCP, DDD, Hexagonal, Onion y Clean

Open-Closed Principle. Martin (2017, p. 70) define el OCP como el principio de que un artefacto debe estar abierto a la extensión pero cerrado a la modificación, y advierte que si extensiones simples de los requisitos obligan a cambios masivos, los arquitectos han fracasado de manera espectacular. El *Kitty Problem* es justamente ese fracaso; la solución con `Rides` y `Kittens` lo corrige: el sistema se extiende añadiendo componentes y filtros sin modificar los existentes.

Adaptadores. `Rides` y `Kittens` cumplen el mismo papel que un adaptador en la Arquitectura Hexagonal (Cockburn, p. 2-3): se conectan a una abstracción definida por el núcleo y aportan una implementación concreta mientras el núcleo permanece ignorante de sus nombres. La selección del componente concreto se realiza en el punto de composición, que son las fábricas controladas por la UI, igual que en la arquitectura de *plugins* que Martin describe para bases de datos e interfaces gráficas (Martin, pp. 170-171).

DDD. El error inicial fue trazar las fronteras según la mecánica del proceso y no según el lenguaje del negocio. Si se hubiese dialogado con expertos del dominio, habría emergido un concepto más general que "viaje", como por ejemplo *servicio de transporte*, del que las distintas modalidades fuesen variantes. Modelar explícitamente ese concepto en el núcleo y dejar las variaciones como extensiones es lo que permite absorber el cambio. Cada contexto delimitado puede así mantener su propio modelo y su propio lenguaje ubicuo sin que un modelo único de "transporte" se contamine con detalles de gatitos o de pasajeros (Evans, 2003; Richards & Ford, 2020, cap. 7).

Onion y Clean. Desde la Arquitectura Cebolla, el núcleo contiene el modelo de dominio y los componentes externos implementan sus interfaces (Palermo, 2008a, p. 2; 2008c, p. 3). Desde Clean, los *interface adapters* convierten datos entre el formato conveniente para el núcleo y el formato del exterior (Martin, p. 205). En ambos casos, la dirección del acoplamiento es hacia el centro y los componentes `Rides` y `Kittens` viven en la periferia, no en el núcleo.

5.7. Lección arquitectónica

Martin (2017, pp. 246-247) concluye que las fronteras arquitectónicas no caen entre servicios, sino que los atraviesan y los dividen internamente en componentes. Los servicios, por útiles que sean para la escalabilidad y el despliegue, no son por sí mismos elementos arquitectónicamente significativos: la arquitectura se define por las fronteras internas y

por las dependencias que las cruzan. La lección del *Kitty Problem* es diseñar cada servicio con una arquitectura interna de componentes que respete la Regla de Dependencia y que permita añadir funcionalidades como extensiones. Separar físicamente no es desacoplar; el desacoplamiento real se consigue invirtiendo las dependencias hacia abstracciones estables del núcleo.

6. Evaluación de la Arquitectura N-Capas frente a Requisitos de Atributos de Calidad

La afirmación del estudiante contiene una parte cierta y otra que confunde una optimización local con una solución arquitectónica. Es cierto que las capas abiertas y el *fast-lane reader* pueden reducir la latencia de determinadas peticiones; sin embargo, no modifican las causas por las que Richards califica a la Arquitectura N-Capas con un nivel bajo en "Agilidad general", "Facilidad de despliegue" y "Escalabilidad" (Richards, 2015, p. 8-9). Estas causas son el carácter monolítico, el acoplamiento entre componentes y la gran granularidad de los despliegues y de la escalabilidad, no la existencia de capas cerradas en sí misma. Por tanto, la propuesta mitiga algunos síntomas de rendimiento, pero no resuelve los atributos de calidad que dice mejorar.

6.1. Qué mitigan realmente las capas abiertas y el *fast-lane reader*

En una arquitectura de capas cerradas, una petición debe atravesar la capa inmediatamente inferior para llegar a la siguiente (Richards, 2015, p. 3). Las capas abiertas son un mecanismo legítimo para permitir que ciertas peticiones omitan una capa intermedia; Richards las presenta, por ejemplo, para que la capa de negocio no se vea obligada a pasar por una capa de servicios compartidos antes de acceder a la persistencia (Richards, 2015, p. 3-4). El *fast-lane reader*, por su parte, es una técnica anterior consistente en que la presentación acceda directamente a la persistencia o a la base de datos para lecturas simples, evitando el recorrido completo por las capas (Richards & Ford, 2020, cap. 10).

Ambas técnicas actúan sobre el *flujo de una petición concreta*, no sobre la estructura de dependencias del sistema. Su alcance real puede resumirse así:

- Mejoran el rendimiento percibido en lecturas simples, al eliminar el costo de atravesar capas que no aportan lógica en ese caso.
- No invierten la dirección de las dependencias: agregan un camino directo desde la presentación hacia la persistencia o la base de datos, con lo cual el acoplamiento aumenta en lugar de disminuir.
- No descomponen el monolito: el sistema sigue siendo una única unidad de despliegue o, en el mejor de los casos, unas pocas unidades físicas, por lo que un cambio pequeño puede seguir exigiendo redesplegar una porción grande de la aplicación (Richards, 2015, p. 8).

- No cambian la granularidad de la escalabilidad: se puede replicar la aplicación completa o separar físicamente algunas capas, pero la unidad de escalado sigue siendo demasiado gruesa y costosa (Richards, 2015, p. 9).
- No introducen modularidad orientada al dominio: la Arquitectura N-Capas es una partición técnica, no una partición por dominio, y un mismo dominio queda repartido entre presentación, negocio, persistencia y base de datos (Richards & Ford, 2020, cap. 10).

En consecuencia, las capas abiertas y el *fast-lane reader* son optimizaciones válidas para un problema acotado de rendimiento, pero no son una respuesta arquitectónica a los requisitos de agilidad, despliegue y escalabilidad. Mitigar un síntoma no equivale a resolver la causa.

6.2. Impacto sobre el Aislamiento de Capas y la mantenibilidad a largo plazo

El principal beneficio de las capas cerradas es el concepto de *layers of isolation*: los cambios realizados en una capa generalmente no impactan en las demás, siempre que los contratos entre ellas se mantengan estables; cada capa es independiente y conoce poco o nada de la implementación interna de las otras (Richards, 2015, p. 3; Richards & Ford, 2020, cap. 10). Ese aislamiento es lo que permite cambiar la tecnología de presentación, la base de datos o un mecanismo de persistencia sin arrastrar al resto del sistema.

Al abrir capas para que la presentación acceda directamente a la persistencia, se pierde precisamente esa garantía. Richards advierte que, si la presentación accede directamente a la persistencia, los cambios realizados en el SQL de esa capa impactan tanto en la capa de negocio como en la de presentación, produciendo una aplicación fuertemente acoplada, con múltiples interdependencias, difícil y costosa de cambiar (Richards, 2015, p. 3). El *fast-lane reader* no elimina ese problema: lo multiplica, porque duplica el conocimiento de la estructura de datos en dos lugares y crea dos caminos de acceso que deben mantenerse coherentes. Las reglas de negocio, la seguridad, las transacciones y la validación pueden terminar aplicándose en un camino y no en el otro, con lo cual aparecen inconsistencias funcionales difíciles de detectar.

Además, la lógica de negocio puede migrar gradualmente hacia la interfaz, ya que el camino rápido evita la capa que debería concentrarla. La arquitectura sigue teniendo carpetas llamadas “presentación”, “negocio” y “persistencia”, pero las fronteras dejan de ser reales: se convierten en una convención que el flujo de las peticiones ya no respeta. A largo plazo, esto incrementa la deuda técnica, vuelve impredecible el impacto de cada cambio y encarece las pruebas y el mantenimiento. El propio Richards señala que no documentar ni comunicar adecuadamente qué capas son abiertas y cerradas, y por qué, suele producir arquitecturas fuertemente acopladas y frágiles, muy difíciles de probar, mantener y desplegar (Richards, 2015, p. 4). La ganancia local de latencia se paga, entonces, con una pérdida global de mantenibilidad.

6.3. Capas abiertas, *sinkhole anti-pattern* y regla 80/20

El *architecture sinkhole anti-pattern* describe la situación en la que las peticiones atraviesan varias capas como simple procesamiento de paso, con poca o ninguna lógica en cada una de

ellas (Richards, 2015, p. 7). Toda arquitectura en capas presenta algunos escenarios de este tipo; lo relevante es su proporción. La regla 80/20 se utiliza como criterio de diagnóstico: lo habitual es que alrededor del 20 % de las peticiones sean simple paso y el 80 % contenga alguna lógica de negocio; si la proporción se invierte y la mayoría de las peticiones son *sinkholes*, es una buena señal de que la Arquitectura N-Capas no es el estilo adecuado para ese dominio (Richards, 2015, p. 7; Richards & Ford, 2020, cap. 10).

Las capas abiertas pueden llevar al *sinkhole anti-pattern* porque suelen proponerse justamente cuando la mayoría de las peticiones no necesitan la lógica de las capas intermedias. Si se abren las capas para que esas peticiones las omitan, no se agrega lógica ni se corrige el problema de fondo: se institucionaliza la ausencia de lógica y se convierte el sistema en un acceso directo de la interfaz a la base de datos, con una capa de negocio meramente decorativa. El *sinkhole* no desaparece; se acelera y se oculta. La petición sigue sin lógica de negocio; solo tarda menos en atravesar el sistema.

Por eso, cuando la regla 80/20 indica que la mayoría de las peticiones son *sinkholes*, la respuesta correcta no es abrir capas, sino cuestionar la elección del estilo arquitectónico y avanzar hacia una partición orientada al dominio, como proponen Hexagonal, Clean u Onion (Richards & Ford, 2020, cap. 10). Abrir capas puede ser aceptable cuando solo una minoría de peticiones es de paso, pero hacerlo como solución general es un *trade-off* que compra velocidad a cambio de aislamiento: el propio Richards recuerda que, al abrir capas, será más difícil controlar el cambio debido a la falta de aislamiento (Richards, 2015, p. 7). La regla 80/20 es un diagnóstico, no una autorización para eliminar las fronteras.

6.4. Por qué Hexagonal y Clean son más adecuadas para alta escalabilidad y agilidad

Conviene precisar el alcance de la afirmación: la Arquitectura Hexagonal y la Arquitectura Limpia no garantizan por sí mismas escalabilidad horizontal ni despliegue independiente; son estilos de organización de dependencias. Su ventaja es que preservan la posibilidad de optimizar, escalar y evolucionar sin contaminar el núcleo, mientras que las capas abiertas de la N-Capas optimizan el camino de una petición a costa de destruir esa posibilidad. Esa diferencia estructural es la que las hace inherentemente más adecuadas para sistemas con altos requisitos de agilidad y escalabilidad.

La Regla de Dependencia exige que las dependencias del código fuente apunten solo hacia adentro, hacia las políticas de mayor nivel, y que nada en un círculo interno sepa algo de un círculo externo (Martin, p. 203). En Hexagonal, la aplicación se comunica con el exterior mediante puertos y cada tecnología externa se conecta mediante un adaptador (Cockburn, p. 3-4). En Clean, la UI y la base de datos pueden ser *plugins* de las reglas de negocio; los detalles dependen de las abstracciones y pueden compilarse y desplegarse como componentes separados (Martin, p. 46-47). Gracias a ello, cambiar SQL Server por MongoDB, introducir una caché, agregar una cola o integrar un servicio externo consiste en crear o sustituir un adaptador que implementa el mismo puerto, sin tocar los casos de uso ni las entidades (Cockburn, p. 3-4; Martin, p. 46-47). A diferencia del *fast-lane*, la optimización no crea un atajo desde la interfaz hacia la base de datos: crea un nuevo adaptador detrás de una abstracción estable.

En términos de agilidad, el núcleo queda aislado de los detalles y puede probarse sin UI ni base de datos real, usando adaptadores falsos o *mocks* (Cockburn, p. 2-3). Los

cambios en un detalle externo se localizan en su adaptador, y los cambios en una regla de negocio se localizan en el caso de uso o la entidad; el impacto deja de ser impredecible. Martin vincula esta separación con la capacidad de desplegar y desarrollar componentes de forma independiente: si los módulos pueden desplegarse por separado, también pueden desarrollarse por equipos distintos (Martin, p. 46-47). Es cierto que un sistema Clean o Hexagonal puede implementarse inicialmente como un único despliegue; la diferencia es que sus fronteras conservan la opción de separar después, mientras que las capas abiertas de la N-Capas no crean costuras, sino atajos.

Respecto de la escalabilidad, el punto decisivo no es que estos estilos repartan mágicamente la carga, sino que permiten aislar y escalar las partes que lo necesitan. Se puede escalar un adaptador de lectura con réplicas, una caché o un servicio de consulta, o agregar un adaptador de mensajería, sin modificar el dominio; también se pueden tener múltiples adaptadores de entrada para una misma aplicación, como la UI web, un proceso batch o un *test harness* (Cockburn, p. 4). En la N-Capas, en cambio, la escalabilidad es de grano grueso: se replica la aplicación completa o se separan físicamente algunas capas, lo que resulta costoso y no permite escalar selectivamente una función del negocio (Richards, 2015, p. 9). Para requisitos altos de escalabilidad, la capacidad de aislar y escalar partes es precisamente lo que marca la diferencia.

Finalmente, la conexión con DDD refuerza la conclusión. La Arquitectura N-Capas es una partición técnica en la que el dominio queda disperso por todas las capas, y por eso el enfoque de Domain-Driven Design no encaja bien con ella (Richards & Ford, 2020, cap. 10). Hexagonal, Clean y Onion, en cambio, mantienen el modelo de dominio en el centro y la infraestructura en los bordes, con las interfaces definidas en las capas internas y las implementaciones en las externas (Palermo, 2008a, p. 2; part 3, p. 3). Esa organización es coherente con la definición de *Bounded Context* de Evans, entendido como la frontera dentro de la cual un modelo particular es definido y aplicable (Evans, 2015, p. 27). Los contextos delimitados aportan el eje de descomposición del dominio y los puertos/adaptadores aportan las costuras para comunicarlos; juntos permiten que la arquitectura acompañe el crecimiento del negocio sin que el núcleo se degrade.

No se trata de una solución gratuita: estos estilos agregan interfaces, indirección, más artefactos y una complejidad inicial mayor, y para aplicaciones pequeñas y simples la N-Capas sigue siendo una opción razonable y de bajo costo (Richards, 2015, p. 9; Richards & Ford, 2020, cap. 10). Pero cuando la agilidad y la escalabilidad son requisitos dominantes, el *trade-off* se invierte: conviene pagar esa complejidad inicial para conservar un núcleo protegido y detalles reemplazables. Las capas abiertas y el *fast-lane reader* optimizan la petición, pero sacrifican el aislamiento; Hexagonal y Clean optimizan la capacidad de cambio y evolución de toda la arquitectura. Esa es la razón por la que, incluso con optimizaciones, una N-Capas sigue siendo estructuralmente menos adecuada para escenarios de alta agilidad y escalabilidad.

Referencias

Cockburn, A. (2005). *Hexagonal Architecture (Ports and Adapters)* [HaT Technical Report 2005.02]. `https://alistair.cockburn.us/hexagonal-architecture/`

De la Torre Llorente, C., Zorrilla Castro, U., Ramos Barroso, M. A., & Calvarro Nelson, J. (2010). *Guía de Arquitectura N-Capas orientada al Dominio con .NET 4.0* (Borrador).

Microsoft Ibérica; Krasis Consulting.

Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

Evans, E. (2015). *Domain-Driven Design Reference: Definitions and Pattern Summaries*. Domain Language.

Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Palermo, J. (2008a, 29 de julio). *The Onion Architecture: Part 1*. https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/

Palermo, J. (2008b, 1 de agosto). *The Onion Architecture: Part 2*. https://jeffreypalermo.com/2008/08/the-onion-architecture-part-2/

Palermo, J. (2008c, 4 de agosto). *The Onion Architecture: Part 3*. https://jeffreypalermo.com/2008/08/the-onion-architecture-part-3/

Richards, M. (2015). *Software Architecture Patterns*. O'Reilly Media.

Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.

Declaración de Uso de la Inteligencia Artificial

Para la elaboración de este seminario, el equipo utilizó la herramienta DeepSeek V4.1 Flash con el propósito de generar el código LaTeX necesario para integrar las respuestas del proyecto y, después de la integración, revisar la correctitud de las referencias bibliográficas. El integrante Adrián fue quien la utilizó para integrar las respuestas en LaTeX. Todo el contenido generado fue revisado, verificado y adaptado por el equipo, quien asume la responsabilidad final del documento.